

TRABAJO PRÁCTICO INTEGRADOR

Asignatura: Programación 3

Docentes: Alberto Cortez.

Cinthia Rigoni.

Alumno: Nicolás Antonio Pannunzio.

Tecnicatura en Programación a Distancia - UTN

Comisión 4 - Cohorte Marzo 2025

ÍNDICE

1. MARCO TEÓRICO.....	1
1.1 Java 17.....	1
1.2 Spring Boot.....	1
1.3 Spring Data JPA Hibernate.....	1
1.4 PostgreSQL.....	1
1.5 Lombok.....	2
1.6 Gradle.....	2
1.7 TypeScript.....	2
1.8 Vite.....	2
1.9 HTML5 y CSS3.....	2
2. DECISIONES TÉCNICAS Y ARQUITECTURA.....	3
2.1 Arquitectura en capas del backend.....	3
2.2 Patrón BaseRepository y Soft Delete.....	4
2.3 Manejo global de excepciones.....	6
2.4 Documentación interactiva con Swagger / OpenApi.....	6
2.5 Patrón DTO.....	8
2.6 Persistencia de datos en PostgreSQL.....	8
2.7 Verificación de endpoints – Postman.....	9
2.8 Transacciones y reducción de stock.....	11
2.9 Autentificación con localStorage.....	12
2.10 Estructura del frontend.....	12
2.11 Interfaces del sistema – Módulo cliente.....	14
2.12 Panel de administración.....	16
2.13 Test.....	19
3. DIFICULTADES Y SOLUCIONES.....	20
3.1 Referencia circular en la serialización JSON.....	20
3.2 Loop de redirección entre páginas.....	21
3.3 Incompatibilidad de versiones Spring Boot y Gradle.....	21
3.4 Datos persistidos en localStorage de versiones anteriores.....	21
3.5 Soft delete en BaseRepository.....	21
4. CONCLUSIONES FINALES.....	22
5. HERRAMIENTAS UTILIZADAS.....	23
6. BIBLIOGRAFÍA.....	24

1. Marco teórico

1.1 Java 17

Java es un lenguaje de programación orientado a objetos, fuertemente tipado y multiplataforma. La versión 17 es una versión LTS (Long Term Support), lo que garantiza soporte extendido y estabilidad. En este proyecto se utilizó como lenguaje principal para el desarrollo del backend, aprovechando características modernas como records, mejoras en switch expressions y la API de streams para programación funcional.

1.2 Spring Boot

Spring Boot es un framework de Java que simplifica la configuración y el desarrollo de aplicaciones. Permite crear aplicaciones autocontenidas con un servidor embebido (Tomcat) sin necesidad de configuración XML. En este proyecto se utilizó para construir la API REST del backend, aprovechando su sistema de inyección de dependencias, configuración automática y soporte nativo para JPA, validaciones y manejo de excepciones.

1.3 Spring Data JPA | Hibernate

JPA (Java Persistence API) es la especificación estándar de Java para el mapeo objeto-relacional (ORM). Hibernate es su implementación más utilizada. Spring Data JPA agrega una capa de abstracción sobre JPA que permite definir repositorios con métodos de consulta automáticos sin escribir SQL manualmente. En este proyecto se utilizó para mapear las entidades del dominio (Usuario, Producto, Categoría, Pedido, DetallePedido) a tablas de la base de datos, y para gestionar las relaciones entre ellas.

1.4 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, conocido por su robustez, soporte de transacciones ACID y amplia compatibilidad con estándares SQL. En este proyecto se utilizó como base de datos principal para persistir todos los datos del sistema, con Hibernate gestionando la creación y actualización del esquema automáticamente mediante la propiedad ddl-auto=update.

1.5 Lombok

Lombok es una herramienta que optimiza el código fuente al eliminar la necesidad de escribir manualmente métodos repetitivos. Actúa como un asistente en tiempo de compilación, inyectando automáticamente la estructura estándar (getters, setters, constructores) en las clases, lo que permite mantener un código más limpio, legible y libre de redundancias. En este proyecto se utilizó extensivamente para simplificar las clases de entidades y DTOs, usando anotaciones como `@Data`, `@Builder`, `@SuperBuilder`, `@NoArgsConstructor`, `@AllArgsConstructor` y `@RequiredArgsConstructor`.

1.6 Gradle

Gradle es una herramienta de automatización de builds que gestiona dependencias, compilación, testing y empaquetado de proyectos. En este proyecto se utilizó como sistema de build del backend, definiendo todas las dependencias en el archivo `build.gradle` y automatizando la ejecución de los tests unitarios y de integración.

1.7 TypeScript

TypeScript es un superconjunto tipado de JavaScript desarrollado por Microsoft. Agrega tipado estático opcional al lenguaje, lo que permite detectar errores en tiempo de compilación en lugar de tiempo de ejecución. En este proyecto se utilizó para desarrollar toda la lógica del frontend, definiendo interfaces para los tipos de datos (`IUser`, `Product`, `ICategory`, `CartItem`) y garantizando la consistencia de los datos intercambiados con la API.

1.8 Vite

Vite es una herramienta de desarrollo frontend moderna que ofrece un servidor de desarrollo ultrarrápido con Hot Module Replacement (HMR) y un proceso de build optimizado. En este proyecto se utilizó como entorno de desarrollo y bundler del frontend, permitiendo trabajar con TypeScript y módulos ES de forma nativa sin configuraciones complejas.

1.9 HTML5 y CSS3

HTML5 es el estándar actual para la estructura de páginas web, mientras que CSS3 es el lenguaje de estilos utilizado para definir la presentación visual. En este proyecto se utilizaron para construir todas las interfaces de usuario del sistema: páginas de autenticación, catálogo de

productos, carrito de compras, historial de pedidos y panel de administración, siguiendo un diseño consistente con variables CSS personalizadas.

2. Decisiones técnicas y Arquitectura

2.1 Arquitectura en capas del backend

El backend fue desarrollado siguiendo una arquitectura en capas claramente definida:

- Controller: Recibe las peticiones HTTP, valida la entrada básica y delega la lógica al servicio correspondiente. No contiene lógica de negocio.
- Service: Contiene toda la lógica de negocio, validaciones de dominio y coordinación entre repositorios.
- Repository: Gestiona el acceso a la base de datos mediante Spring Data JPA.
- DTO (Data Transfer Object): Objetos intermedios que desacoplan la representación de las entidades internas del contrato de la API pública.

Esta separación garantiza que cada capa tenga una única responsabilidad, facilitando el mantenimiento y la escalabilidad del sistema.

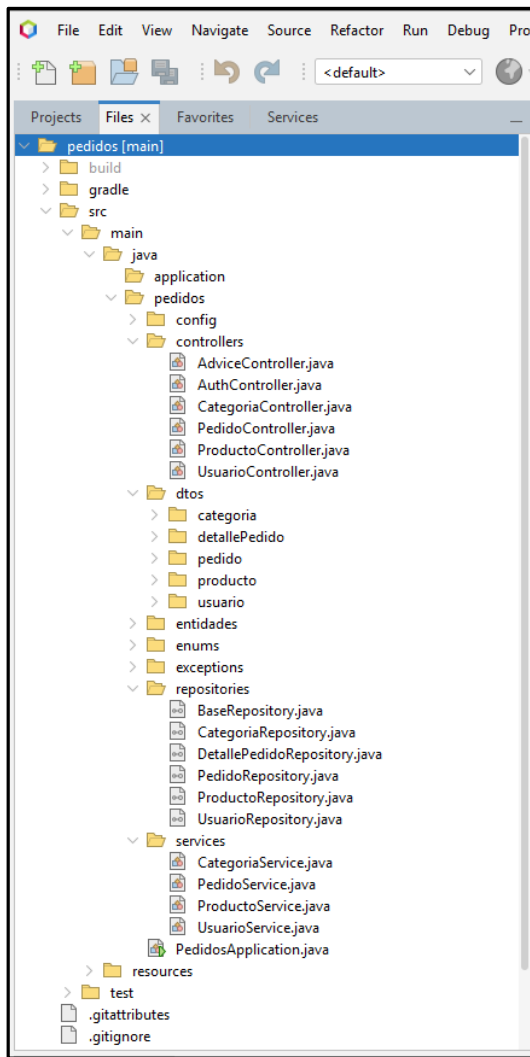


Figura 1: Estructura de paquetes y organización de capas del backend en el proyecto.

2.2 Patrón BaseRepository y Soft Delete

Una de las decisiones más importantes fue la implementación de una clase base Base y una interfaz BaseRepository genérica que centralizan comportamientos comunes a todas las entidades.

La clase Base define campos compartidos por todas las entidades:

id: identificador autogenerado

eliminado: flag para soft delete lógico

createdAt: timestamp de creación automático

updatedAt: timestamp de última modificación automático

version: campo para optimistic locking

El BaseRepository extiende JpaRepository y sobrescribe el comportamiento de los métodos principales para que el soft delete sea transparente:

findAll() retorna solo entidades con eliminado = false

findByIdOrThrow() lanza ResourceNotFoundException si no encuentra la entidad

softDelete() marca la entidad como eliminada sin borrarla físicamente

Esta decisión evita la duplicación de código en todos los repositorios y garantiza que ninguna entidad sea eliminada físicamente de la base de datos, preservando el historial de datos.

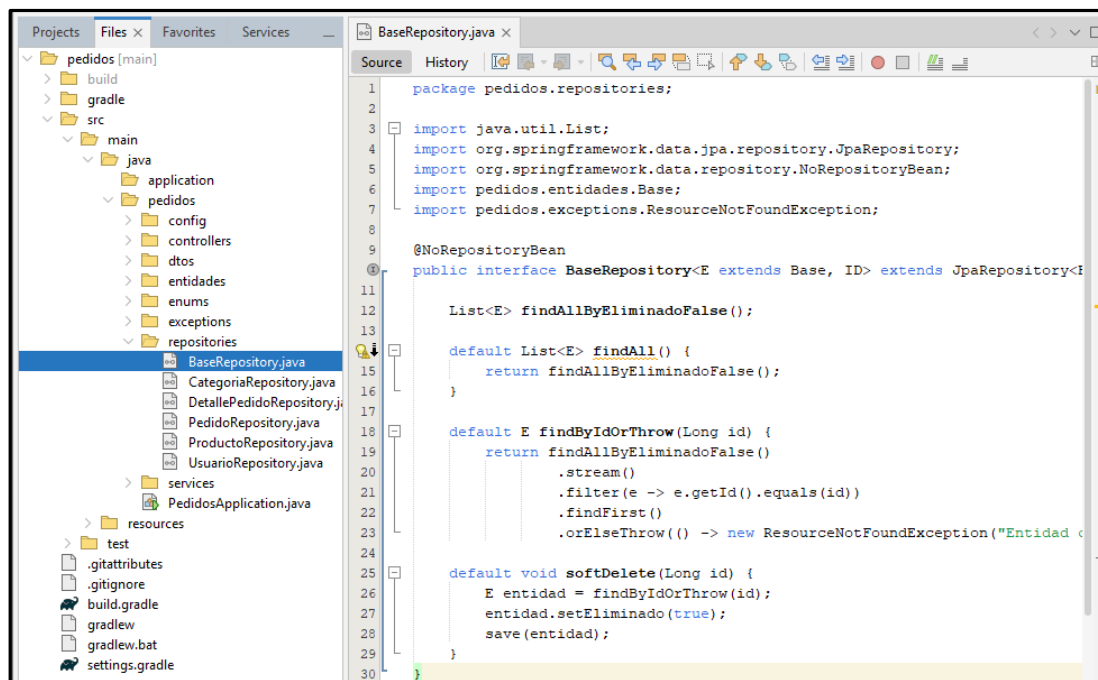


Figura 2: Implementación del patrón BaseRepository para la centralización de lógica de persistencia y gestión de soft delete.

2.3 Manejo global de excepciones

Se implementó un `GlobalExceptionHandler` con la anotación `@RestControllerAdvice` que centraliza el manejo de errores en un único lugar. Esto garantiza que todas las respuestas de error de la API tengan siempre el mismo formato JSON:

```
{  
  
    "status": 404,  
  
    "mensaje": "Entidad con id 999 no encontrado",  
  
    "timestamp": "2026-06-17T12:00:00"  
}
```

Los tipos de excepciones manejados en este proyecto son:

`ResourceNotFoundException` → HTTP 404

`BusinessException` → HTTP 400

`MethodArgumentNotValidException` → HTTP 400 con detalle de campos

Excepción genérica → HTTP 500

2.4 Documentación interactiva con Swagger / OpenApi

Se integró SpringDoc OpenAPI para generar documentación automática e interactiva de todos los endpoints de la API REST. Cada controlador fue anotado con `@Tag` para agruparlo temáticamente, y cada endpoint con `@Operation` describiendo su propósito, lo que permite explorar y probar la API directamente desde el navegador sin necesidad de herramientas externas.



Figura 3: Vista general de los endpoints de la API agrupados por entidad en Swagger UI.

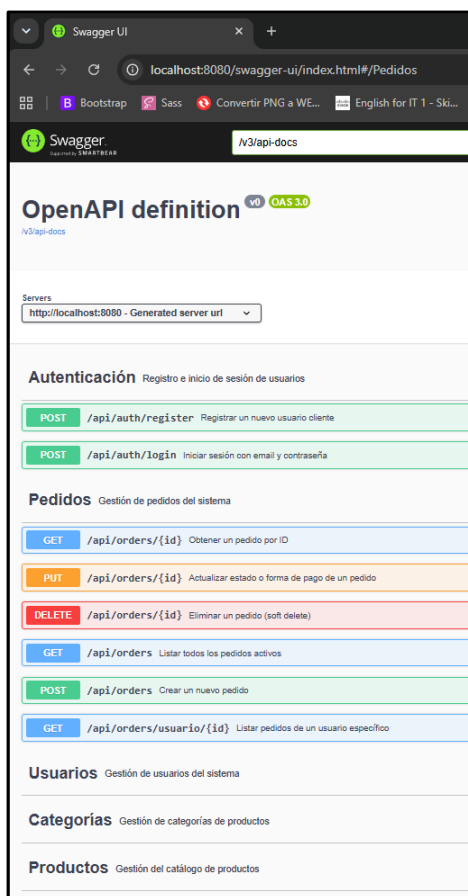


Figura 4: Detalle de los endpoints del módulo de Pedidos, incluyendo método HTTP y descripción de cada operación.

2.5 Patrón DTO

Se utilizaron DTOs para separar la representación interna de las entidades del contrato público de la API. Cada entidad tiene hasta tres DTOs asociados:

Create: campos requeridos para crear una nueva instancia

Edit: campos opcionales para actualizar (todos nullable para soportar actualización parcial)

Dto: representación de respuesta, omitiendo campos sensibles como contraseñas

Esta decisión protege la integridad del modelo de dominio y evita exponer información innecesaria o sensible al cliente.

2.6 Persistencia de datos en PostgreSQL

Para verificar la correcta integración entre la capa de persistencia y la base de datos PostgreSQL, se utilizó DBeaver como herramienta de gestión. A continuación, se detalla la estructura relacional generada automáticamente por Hibernate a partir de las entidades del sistema, confirmando que el esquema de datos es consistente con el diseño propuesto.

	id	created_at	eliminado	updated_at	version	AZ	apellido
1	1	2026-06-13 12:26:37.363	[]	2026-06-13 12:26:37.363	0	Sistema	
2	2	2026-06-13 12:38:24.556	[]	2026-06-13 12:38:24.556	0	Perez	
3	3	2026-06-13 15:32:23.223	[]	2026-06-13 15:32:23.223	0	rita	
4	4	2026-06-13 15:43:20.029	[]	2026-06-13 15:43:20.029	0	mar	
5	5	2026-06-16 17:53:26.289	[]	2026-06-16 17:53:26.289	0	nico	

Figura 3: Esquema de tablas generado automáticamente por Hibernate en PostgreSQL.

The screenshot shows the DBeaver 26.0.1 interface. On the left, the 'Navegador de Bases de Datos' (Database Navigator) displays a tree structure for a PostgreSQL database named 'delivery_app_p3'. The 'public' schema is expanded, showing tables: 'categoria' (32K), 'detalle_pedido' (24K), 'pedido' (32K), 'producto' (32K), and 'usuario' (32K). The main editor shows a SQL script with the query 'select * from pedido;'. Below the editor, the 'pedido 1' table is displayed in a grid view, showing 17 rows of data. The status bar at the bottom indicates '17 row(s) fetched - 0,004s (0,001s fetch), on 2026-06-21 at 16:43:14'.

	id	created_at	eliminado	updated_at	version	estado
1	1	2026-06-13 12:39:07.007	[]	2026-06-13 12:39:07.007	0	PENDIENTE
2	2	2026-06-13 15:44:14.672	[]	2026-06-13 15:44:14.672	0	PENDIENTE
3	3	2026-06-13 15:55:37.531	[]	2026-06-13 15:55:37.531	0	PENDIENTE
4	4	2026-06-13 16:05:34.298	[]	2026-06-13 16:05:34.298	0	PENDIENTE
5	5	2026-06-13 16:13:54.919	[]	2026-06-13 16:13:54.919	0	PENDIENTE
6	6	2026-06-13 18:49:25.050	[]	2026-06-13 18:49:25.050	0	CONFIRMADO
7	7	2026-06-13 18:49:57.326	[]	2026-06-13 18:49:57.326	0	CANCELADO
8	8	2026-06-13 18:53:58.512	[]	2026-06-13 18:53:58.512	0	PENDIENTE
9	9	2026-06-13 19:13:23.410	[]	2026-06-13 19:13:23.410	0	PENDIENTE
10	10	2026-06-13 19:28:29.661	[]	2026-06-13 19:28:29.661	0	PENDIENTE
11	11	2026-06-13 19:31:39.970	[]	2026-06-13 19:31:39.970	0	PENDIENTE
12	12	2026-06-13 19:35:53.597	[]	2026-06-13 19:35:53.597	0	PENDIENTE
13	13	2026-06-13 19:45:03.418	[]	2026-06-13 19:45:03.418	0	PENDIENTE
14	14	2026-06-13 19:49:47.037	[]	2026-06-13 19:49:47.037	0	PENDIENTE
15	15	2026-06-13 19:59:22.707	[]	2026-06-13 19:59:22.707	0	PENDIENTE

Figura 4: Resultado de la consulta SQL SELECT * FROM pedido, verificando la persistencia efectiva de registros reales en la base de datos PostgreSQL tras las operaciones de gestión.

2.7 Verificación de endpoints – Postman

Durante el desarrollo del backend se utilizó Postman para verificar de forma manual el comportamiento de cada endpoint antes de integrarlo con el frontend. Esto permitió validar tanto los casos exitosos como las respuestas de error generadas por las reglas de negocio implementadas, como el control de stock insuficiente o la validación de recursos inexistentes.

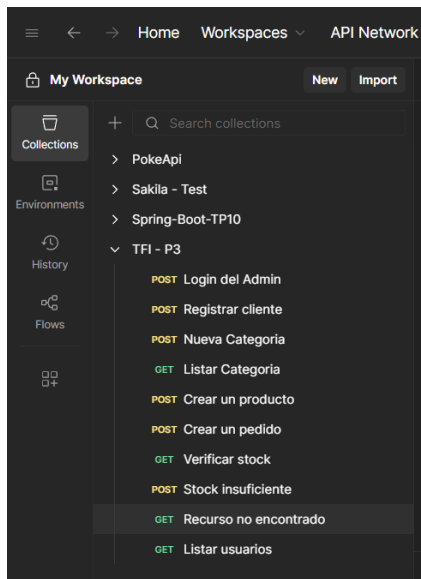


Figura 5: Colección de endpoints en Postman utilizada para validar el comportamiento de la API REST.

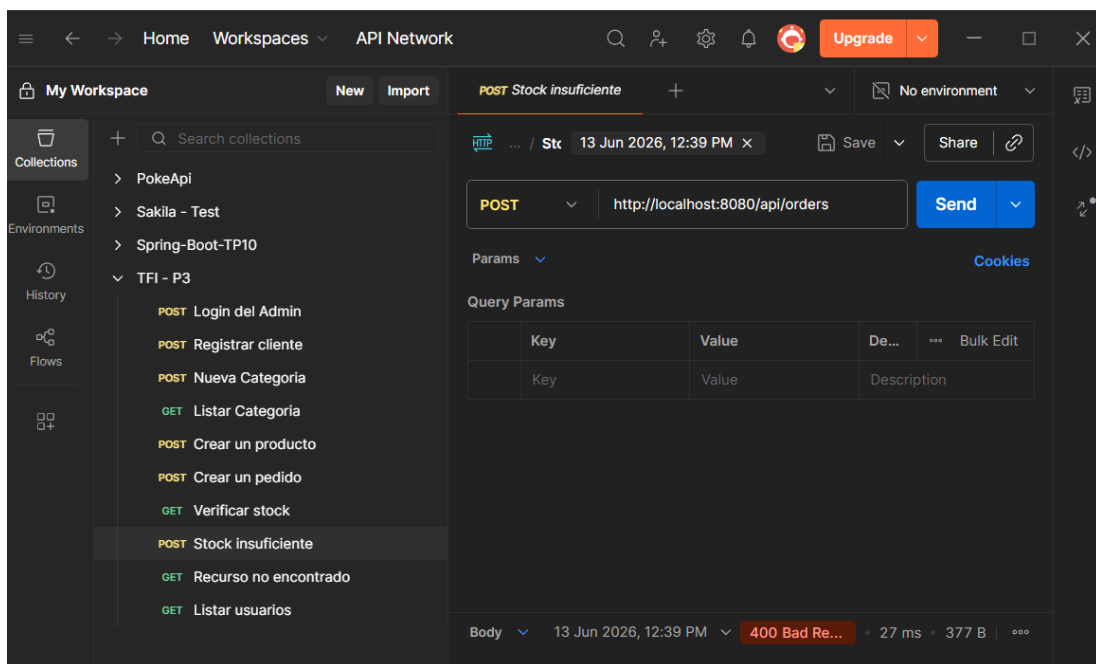


Figura 6: Validación de regla de negocio — respuesta 400 Bad Request al solicitar una cantidad superior al stock disponible.

2.8 Transacciones y reducción de stock

La creación de pedidos es la operación más crítica del sistema. Se implementó con `@Transactional(rollbackOn = Exception.class)` para garantizar que si alguna validación falla durante el procesamiento, todas las operaciones previas se revierten automáticamente.

El flujo de creación de un pedido es:

- Validar que el usuario existe
- Validar que hay al menos un producto en el detalle
- Para cada producto: verificar disponibilidad y stock suficiente
- Reducir el stock de cada producto
- Calcular subtotales y total del pedido
- Persistir el pedido con todos sus detalles

Si en el paso 3 o 4 falla algún producto, el rollback revierte todas las modificaciones de stock realizadas hasta ese momento.

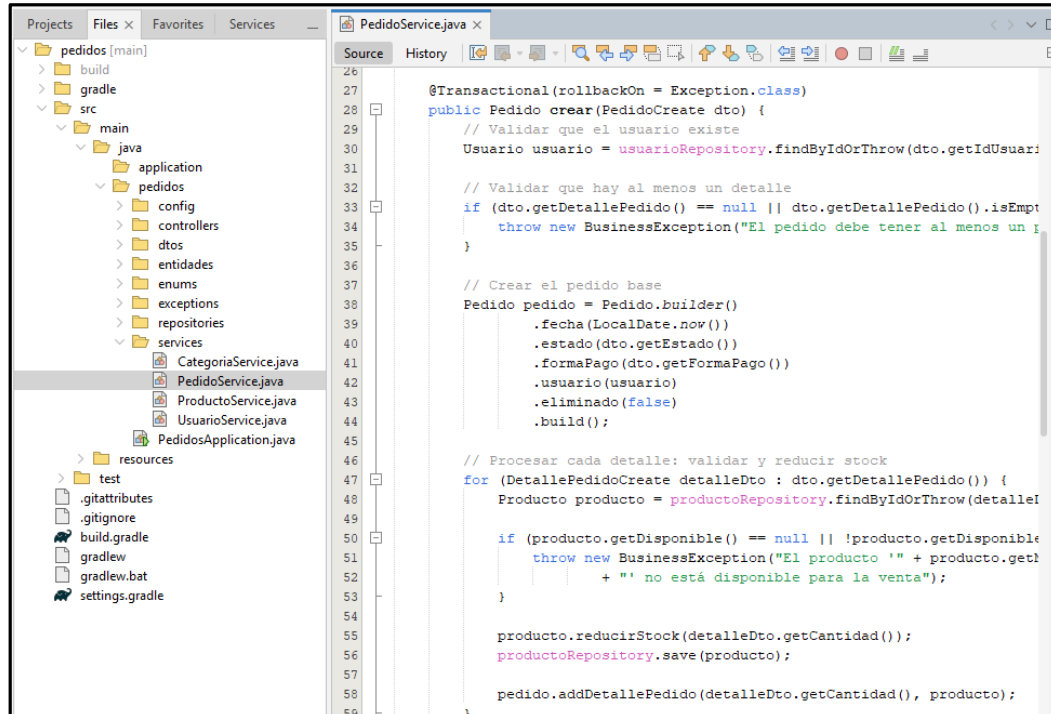


Figura 7: Implementación del servicio de creación de pedidos, aplicando el manejo de transacciones `@Transactional` para garantizar la integridad y consistencia de los datos en el inventario.

2.9 Autenticación con localStorage

Por indicación en las rúbricas para el desarrollo de este proyecto, la autenticación se implementó de forma simplificada usando localStorage del navegador, sin tokens JWT.

El flujo es:

- El usuario envía sus credenciales al endpoint /api/auth/login
- El backend valida las credenciales y retorna los datos del usuario incluyendo su rol
- El frontend guarda el objeto usuario en localStorage bajo la clave userData
- Cada página protegida verifica la existencia y el rol del usuario en localStorage al cargarse
- Si no hay sesión o el rol no corresponde, redirige al login

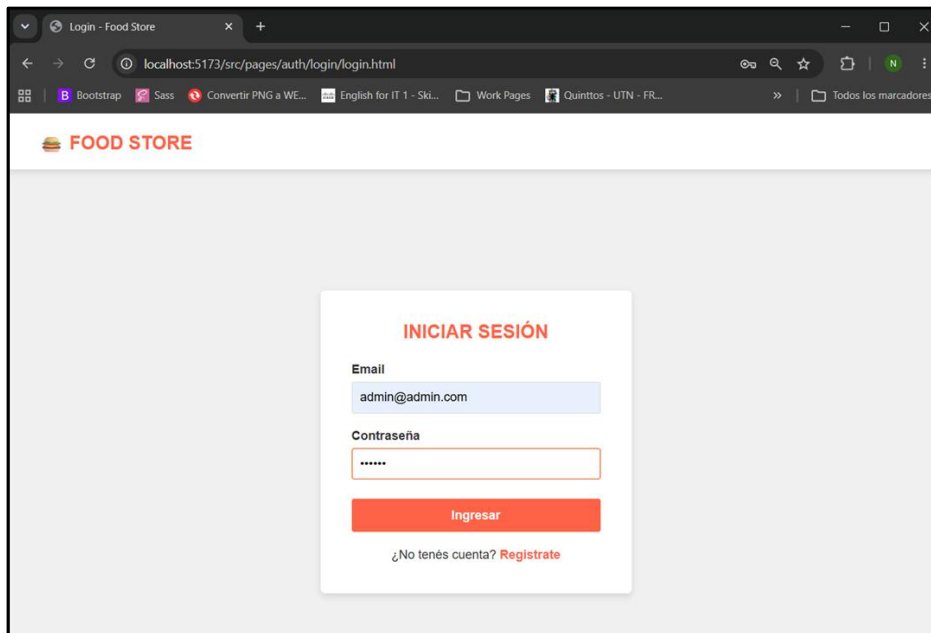


Figura 8: Pantalla de login compartida entre clientes y administradores; el rol determina la redirección.

2.10 Estructura del frontend

El frontend fue organizado siguiendo la estructura de carpetas propuesta por el TPI.

Un componente clave de esta organización es el archivo api.ts (ubicado en src/utills), ya que centraliza todas las llamadas HTTP al backend, evitando repetir la URL base y el manejo de errores en cada componente. La distribución de estas responsabilidades se detalla a continuación:

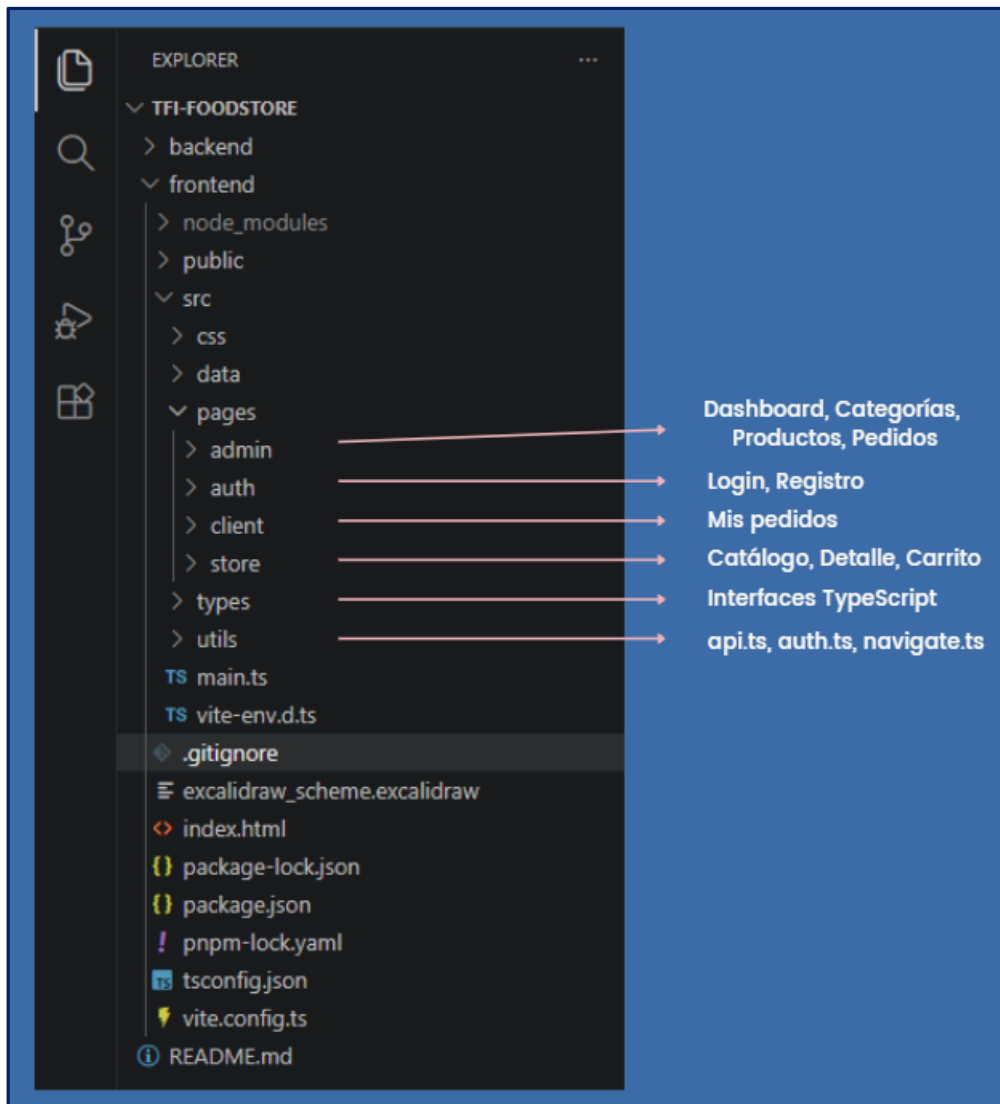


Figura 9: Organización modular del proyecto frontend, detallando la distribución de páginas, interfaces de TypeScript y utilidades de navegación.

2.11 Interfaces del sistema – Módulo cliente

FOOD STORE

CREAR CUENTA

Nombre
Tu nombre

Apellido
Tu apellido

Email
tucorreo@mail.com

Celular
Tu número de celular

Contraseña
Mínimo 6 caracteres

Registrarse

¿Ya tenés cuenta? [Inicia sesión](#)

Figura 10: Pantalla de registro de usuario, con validación de campos obligatorios.

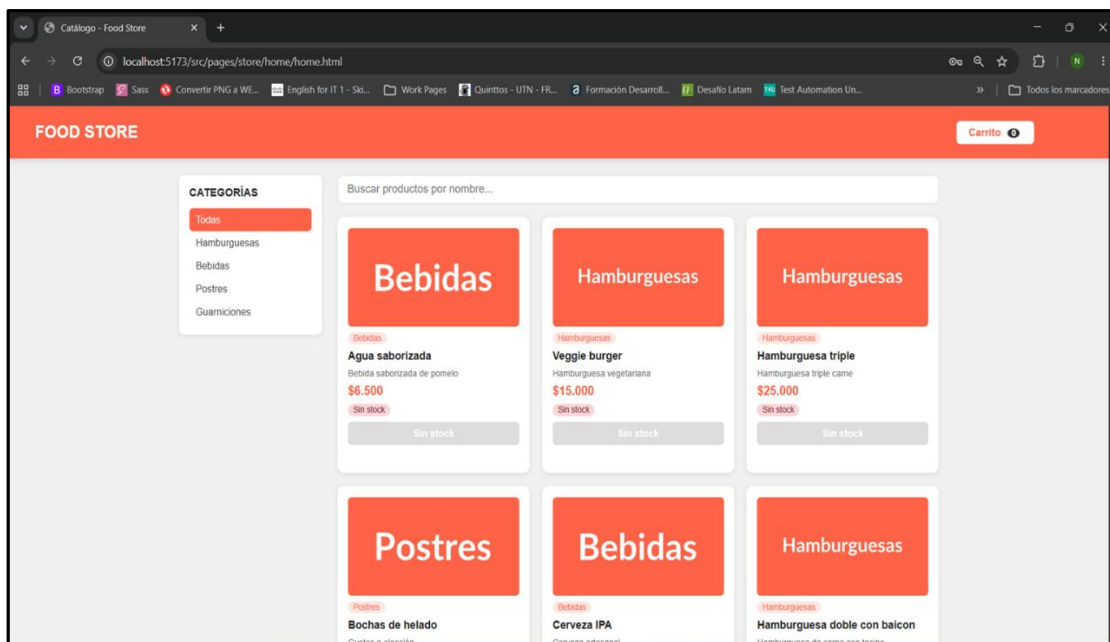


Figura 11: Catálogo de productos y grilla de productos disponibles.

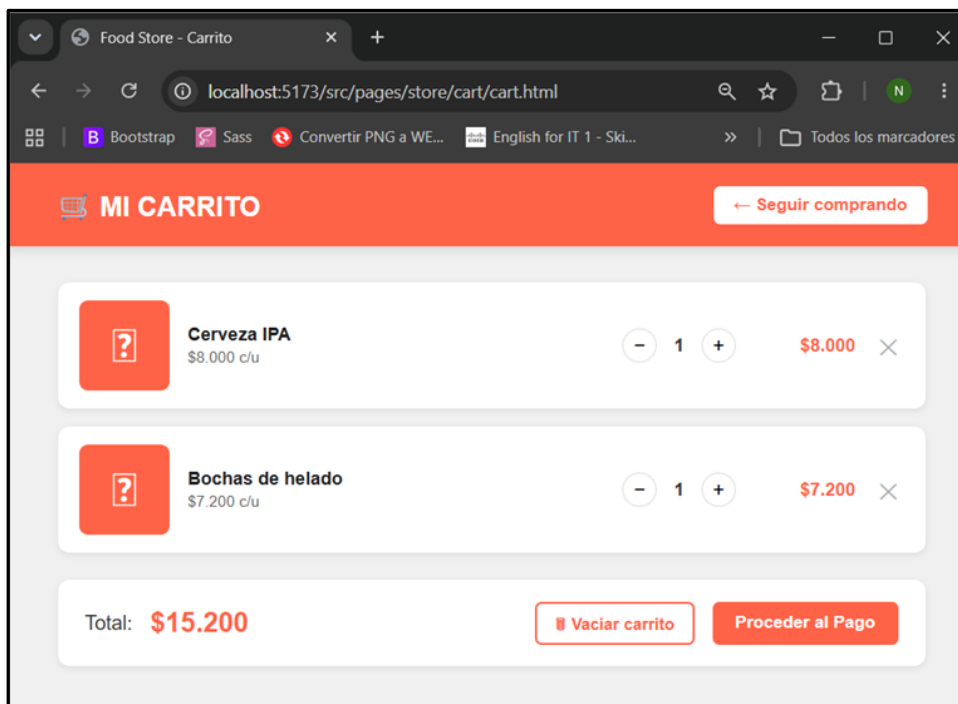


Figura 12: Carrito de compras con control de cantidades, subtotal por producto y total general.

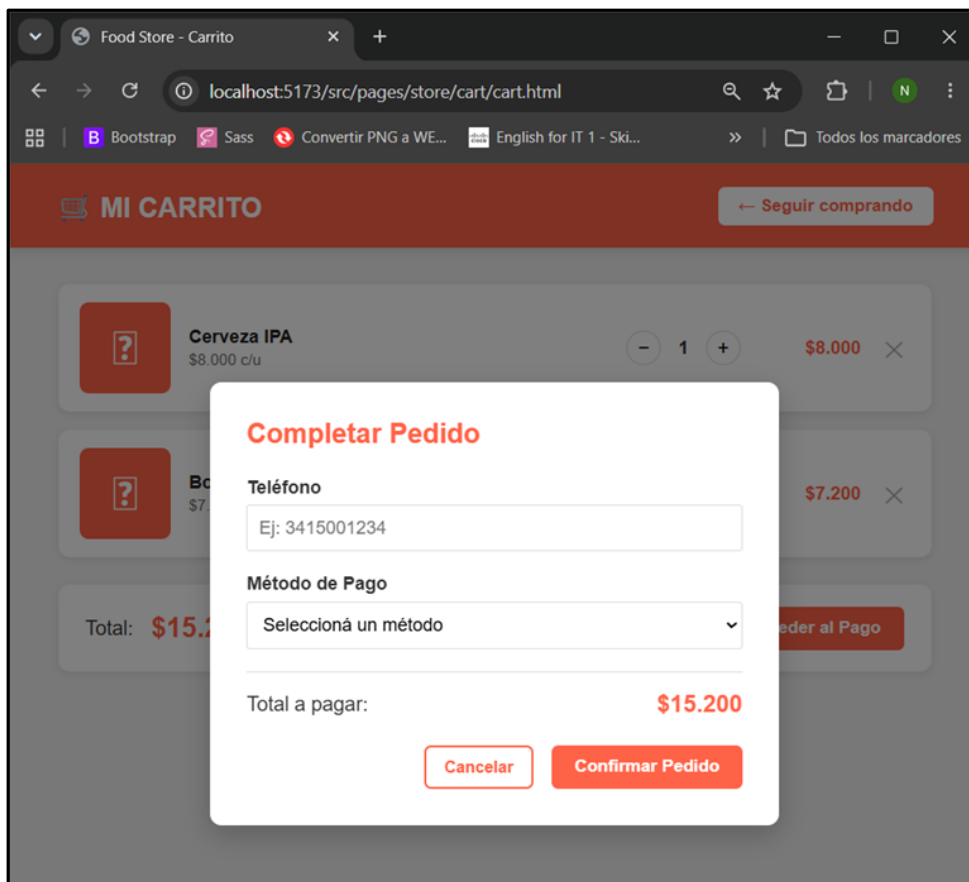


Figura 13: Modal de checkout solicitando teléfono y método de pago antes de confirmar el pedido.

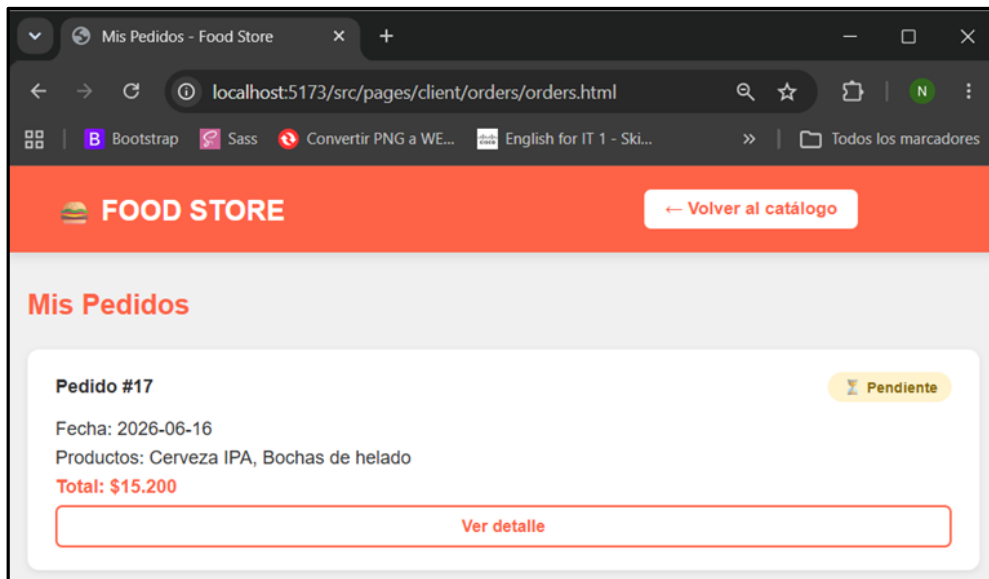


Figura 14: Historial de pedidos del cliente autenticado, mostrando estado y resumen de cada orden.

2.12 Panel de administración

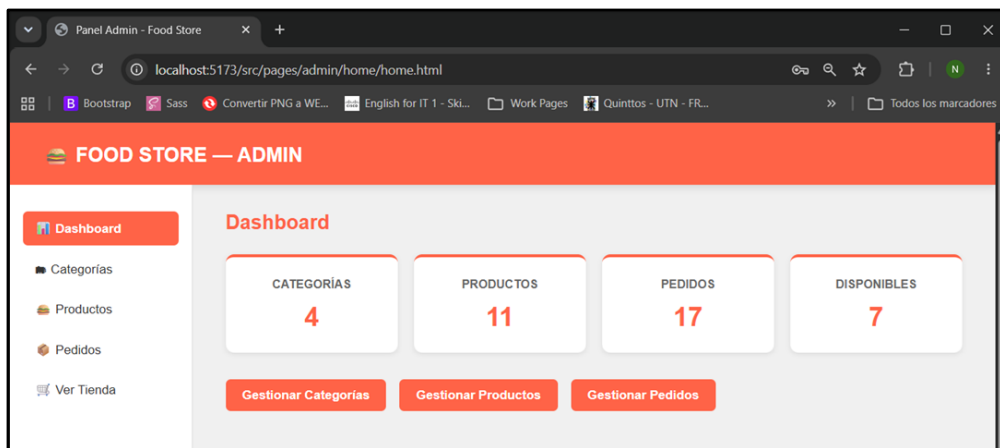


Figura 15: Vista principal del dashboard administrativo, mostrando métricas de gestión del sistema.

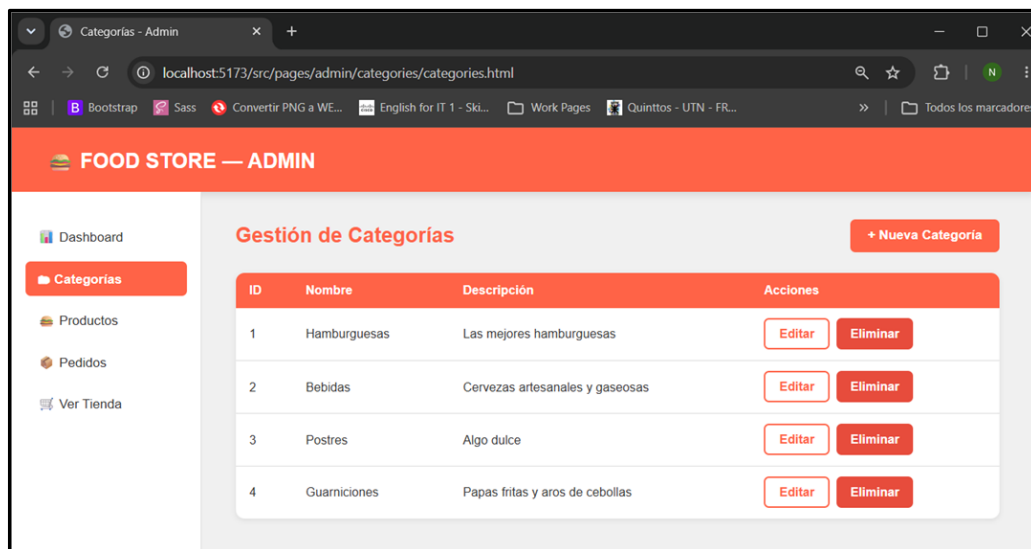


Figura 16: Interfaz de gestión de categorías, permitiendo la edición y eliminación de registros.

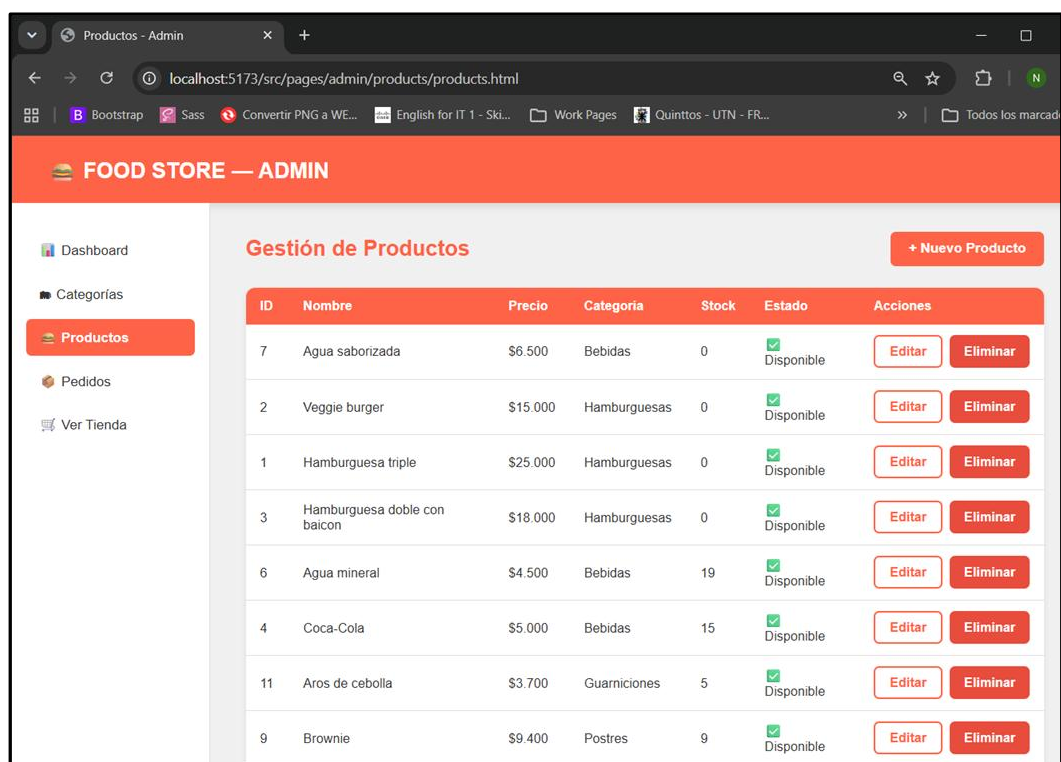


Figura 17: Gestión de productos desde el panel admin, con tabla CRUD y estado de disponibilidad.

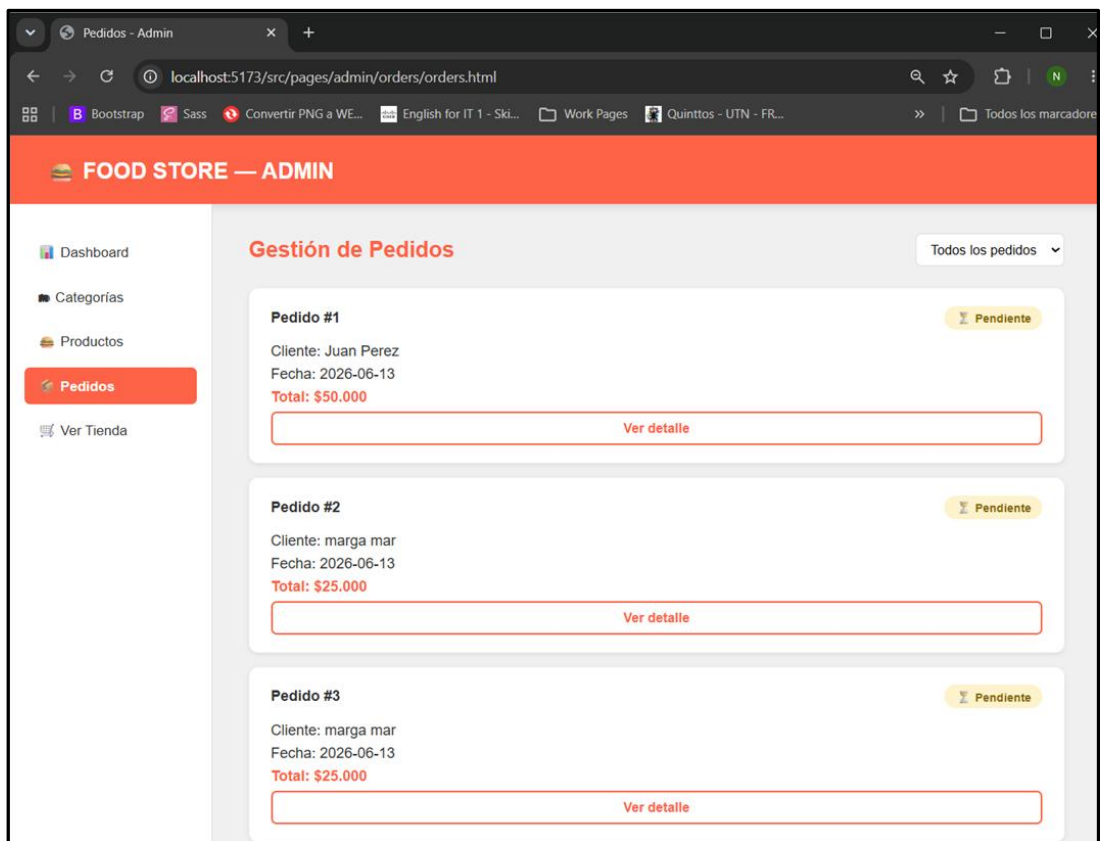


Figura 18: Gestión de pedidos, permitiendo visualizar el historial completo y actualizar estados.

2.13 Test

Se implementaron dos tipos de tests:

Tests unitarios usando JUnit 5 y Mockito, que verifican la lógica de negocio de cada servicio de forma aislada, simulando los repositorios con mocks. Se cubren escenarios exitosos y de error para `CategoriaService`, `ProductoService`, `UsuarioService` y `PedidoService`.

Tests de integración usando `@SpringBootTest` y `MockMvc`, que levantan el contexto completo de Spring con una base de datos H2 en memoria y verifican el comportamiento real de los endpoints REST de categorías, incluyendo códigos de respuesta HTTP y estructura del JSON retornado.

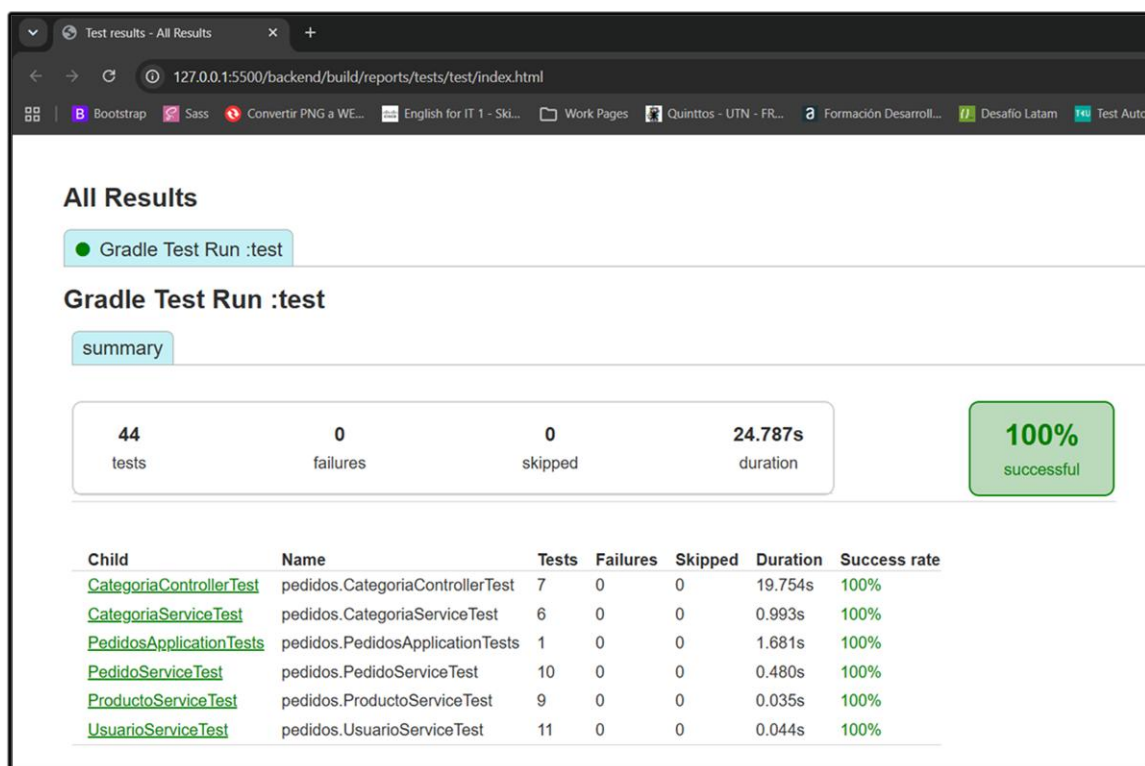


Figura 19: Informe consolidado de ejecución de pruebas automatizadas, confirmando una tasa de éxito del 100% sobre la suite completa de 44 casos de prueba.

Se implementaron en total 44 tests, distribuidos entre tests unitarios y tests de integración, obteniendo una tasa de éxito del 100%:

Clase	Tipo	Cantidad
CategoriaControllerTest	Integración	7
CategoriaServiceTest	Unitario	6
PedidosApplicationTests	Contexto	1
PedidoServiceTest	Unitario	10
ProductoServiceTest	Unitario	9
UsuarioServiceTest	Unitario	11
Total		44

3. Dificultades y soluciones

A lo largo del desarrollo del TFI, me enfrenté a diversos desafíos técnicos que pusieron a prueba mi capacidad de resolución de problemas. A continuación, detallo algunos de los obstáculos más significativos encontrados durante la implementación, junto con las soluciones específicas que apliqué para garantizar la estabilidad y funcionalidad del sistema.

3.1 Referencia circular en la serialización JSON

Problema: Al serializar un Pedido a JSON, Hibernate intentaba serializar también el Usuario asociado, que a su vez tenía una lista de Pedidos, generando un loop infinito que superaba la profundidad máxima permitida por Jackson (500 niveles).

Solución: Se aplicó la anotación `@JsonIgnore` sobre la referencia al Pedido dentro de `DetallePedido`, cortando el ciclo de serialización. Esto permite que el pedido incluya sus detalles con la información del producto, sin generar el loop.

3.2 Loop de redirección entre páginas

Problema: Al implementar la protección de rutas en el frontend, se generó un loop infinito de redirecciones. La página de pedidos del cliente redirigía al home del store, que a su vez redirigía de vuelta, repitiendo el ciclo indefinidamente en la URL del navegador.

Solución: Se identificó que existían dos archivos `home.ts` en conflicto — uno en `src/pages/client/home/` con lógica de redirección incorrecta y otro en `src/pages/store/home/` con el catálogo real. Se eliminó la carpeta `client/home/` que era redundante y se unificó la estructura de carpetas para que `client/` solo contenga `orders/`.

3.3 Incompatibilidad de versiones Spring Boot y Gradle

Problema: El proyecto fue iniciado con Spring Boot 4.0.6, una versión muy reciente con cambios en la ubicación de clases de testing que generaron errores al intentar usar `@AutoConfigureMockMvc`. Además, Spring Boot 3.3.0 resultó incompatible con Gradle 9.5.1.

Solución: Se migró a Spring Boot 3.4.5, versión estable compatible con Gradle 9.x que mantiene `@AutoConfigureMockMvc` en su ubicación original y resolvió todos los errores de testing.

3.4 Datos persistidos en localStorage de versiones anteriores

Problema: Durante el desarrollo, el frontend anterior guardaba usuarios en `localStorage` con un formato distinto al que devuelve el backend. Al probar el nuevo sistema de login, los datos viejos en `localStorage` causaban que el usuario autenticado no tuviera `id`, lo que generaba errores al crear pedidos.

Solución: Se limpió el `localStorage` desde las herramientas de desarrollador del navegador y se verificó que el nuevo formato de `userData` incluyera todos los campos necesarios provenientes del backend.

3.5 Soft delete en BaseRepository

Problema: Al implementar el `BaseRepository` genérico, el método `deleteById` heredado de `JpaRepository` realizaba una eliminación física en lugar de lógica, lo que contradecía el diseño del sistema.

Solución: Se implementó un método `softDelete()` en `BaseRepository` que busca la entidad por ID y establece `eliminado = true` antes de guardarla, sobreescribiendo el comportamiento por defecto de JPA de forma transparente para todos los servicios.

4. Conclusiones finales

El desarrollo del presente Trabajo Práctico Integrador me permitió consolidar los conocimientos adquiridos a lo largo de la cursada, integrando de manera efectiva el ciclo de vida completo de una aplicación web. A continuación, presento las valoraciones principales de esta experiencia:

- Cumplimiento de objetivos: Se logró implementar con éxito un sistema full stack funcional, cumpliendo con todos los requerimientos funcionales y no funcionales propuestos por la cátedra. La aplicación permite una gestión integral de productos, usuarios y pedidos, con una arquitectura robusta y testeada.
- Integración de conocimientos técnicos: Este proyecto fue el escenario ideal para aplicar de forma concreta tecnologías clave como JPA/Hibernate para la persistencia de datos, TypeScript para la lógica del frontend, y un robusto ecosistema Spring Boot para el backend. Además, la integración de Swagger/OpenAPI permitió documentar y probar de forma eficiente cada servicio.
- Aprendizajes clave: Más allá de la implementación técnica, destaco el aprendizaje obtenido en el manejo real de problemas complejos: la resolución de referencias circulares en la serialización, el manejo correcto de transacciones para garantizar la integridad del inventario y la superación de conflictos de configuración entre dependencias de build tools.
- Valoración personal: Construir un sistema desde cero —desde el diseño de la base de datos hasta el despliegue del panel de administración— me brindó una visión clara de los desafíos reales que enfrenta un desarrollador. La adopción de buenas prácticas de testing y la centralización de lógica de negocio (como el patrón BaseRepository o el manejo global de excepciones) han sido fundamentales para garantizar la calidad y mantenibilidad del producto final.

5. Herramientas utilizadas

Tecnología	Categoría	Funcionalidad
Netbeans	IDE	Entorno de desarrollo backend
VSCode	Editor	Entorno de desarrollo frontend
Java 17	Lenguaje	Backend
Spring Boot 3.4.5	Framework	API REST
Gradle	Build tool	Gestión de dependencias y tests
PostgreSQL	Base de datos	Persistencia
DBeaver	Cliente DB	Verificación de datos
Postman	Cliente API	Testing manual de endpoints
TypeScript	Lenguaje	Frontend
Vite	Bundler	Entorno de desarrollo frontend
Git / GitHub	Control de versiones	Repositorio del proyecto
Swagger / OpenAPI	Documentación	Documentación interactiva de la API
JUnit 5 / Mockito	Testing	Test unitarios y de integración

6. Bibliografía

Spring Boot — Documentación oficial: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring Data JPA — Documentación oficial: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>

Hibernate ORM — Documentación oficial: <https://hibernate.org/orm/documentation/>

Lombok — Documentación oficial: <https://projectlombok.org/>

SpringDoc OpenAPI — Documentación oficial: <https://springdoc.org/>

TypeScript — Documentación oficial: <https://www.typescriptlang.org/docs/>

Vite — Documentación oficial: <https://vitejs.dev/guide/>

JUnit 5 — Documentación oficial: <https://junit.org/junit5/docs/current/user-guide/>

Mockito — Documentación oficial: <https://site.mockito.org/>

PostgreSQL — Documentación oficial: <https://www.postgresql.org/docs/>